



Machine Learning [CSE374s]

Credit Card Default (Taiwan)

Member Name	ID
Abdelrahman Hany Ashraf (role 1)	2300130
Ahmed Ayman Mohamed (role 2)	2300145
Yassin Ayman Elsayed (role 3)	2301087

Introduction

The Problem: Credit Card Default

Many banks face a big problem: customers who take credit card loans but cannot pay them back. This is called "defaulting." For any bank, it is very important to predict if a customer will pay or not. Knowing the probability of a customer defaulting is actually more useful for the bank than just a "Yes" or "No" answer, as it helps them manage risks better.

Our Project Goal

In this project, we are using machine learning to solve this problem. We are building a model that looks at customer data to predict if they will pay their credit card bill next month or not.

Classification

Classification in machine learning can be "multiclass" (many categories) or "binary" (only two categories). In our case, we use **Binary Classification**. This means the output will only be one of two things:

1. **(1)**: The customer will **not** pay next month.
2. **(0)**: The customer **will** pay next month.

Data Analysis and Preprocessing

The dataset contains 30,000 clients and 23 features. However, real-world data often contains inconsistencies, missing values, and noise. Training a model on raw, "dirty" data leads to poor performance and low accuracy. To ensure high-quality predictions, we performed the following preprocessing steps:

1. Data Type Consistency

Machine learning models require numerical inputs for mathematical calculations. We ensured that all features were converted to **float64**. This consistency allows the model to process all values correctly without errors during the training phase.

2. Duplicate and Missing Value Handling

We checked the dataset for any duplicate entries to avoid redundant information, but no duplicates were found. For missing or undocumented values, specifically in the "Education" and "Marriage" features, we identified rows with values not defined in the dataset description. Since these represented only **1.3%** of the total data, we removed them without significantly affecting the dataset size.

```
df = df[df['X3'] != 6]
df = df[df['X3'] != 5]
df = df[df['X3'] != 0]

df['X3'].value_counts() #removed all values (0,5,6)

...
X3
2.0    14030
1.0    10585
3.0     4917
4.0     123
Name: count, dtype: int64

df = df[df['X4'] != 0]

df['X4'].value_counts() #removed all values (0)

...
X4
2.0    15806
1.0    13477
3.0     318
Name: count, dtype: int64
```

3. Outlier Management (Capping)

We identified outliers in features like credit limits and payment amounts. Instead of deleting these records—which would lose valuable information—we used a **Capping** strategy. Any value exceeding the statistical upper bound was set to that maximum limit. This prevents extreme values from distorting the model's learning process.

4. Handling Categorical Variables (One-Hot Encoding)

Features such as Gender, Education, and Marital Status are categorical. Assigning them simple numbers (like 1, 2, 3) can mislead the model into thinking there is a ranking or order (e.g., that "Female" is higher than "Male"). To solve this, we used **One-Hot Encoding**. This technique creates independent columns for each category, ensuring the model treats them as distinct labels without any bias or ranking.

5. Addressing Class Imbalance (SMOTE)

Our dataset suffered from a significant class imbalance: **77.68%** of the clients were non-defaulters (0), while only **22.32%** were defaulters (1). This imbalance

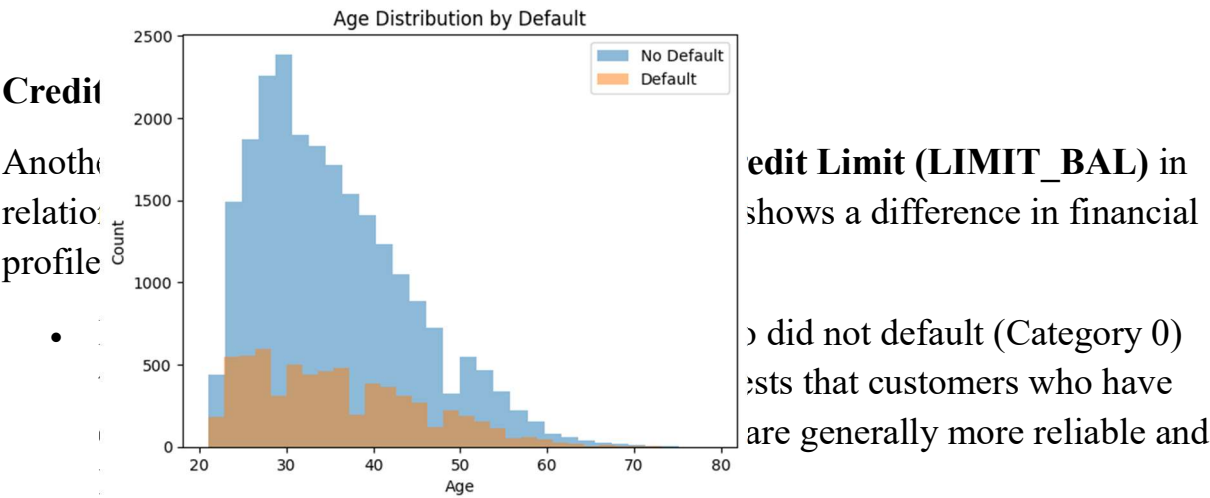
can cause the model to be biased toward the majority class. To fix this, we applied the **SMOTE (Synthetic Minority Over-sampling Technique)**. SMOTE uses the k-Nearest Neighbors (kNN) algorithm to generate synthetic (artificial) samples for the minority class, ensuring the model learns to identify potential defaulters effectively.

Exploratory Data Analysis (EDA)

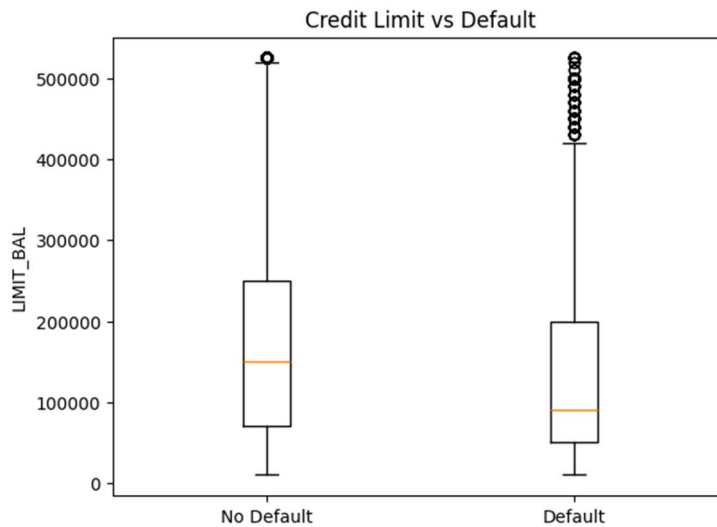
Before training the model, we analyzed the distribution of our features to understand the customer base. One of the most important features is **Age (X5)**.

Key Findings from Age Distribution:

- **Age Concentration:** The majority of credit card users are young adults, with the largest group falling between the ages of **25 and 35**. This indicates that the bank's primary customer base is in the early-to-mid career stage.
- **Defaulting Trends:** While defaulting occurs across all age groups, it is more visible in the younger section of the chart. This is because there are more young people in the dataset overall, leading to a higher number of "Default" cases (the orange bars) in that age range.
- **Usage in Older Age Groups:** As customers get older (specifically past age **50**), the number of both "Good Payers" and "Defaulters" drops significantly. This shows that older individuals in this dataset use credit cards less frequently or are fewer in number compared to younger generations.



- **Default Behavior:** On the other hand, the data shows that default behavior (Category 1) is more frequently associated with lower credit limits. This could indicate that higher-risk customers are restricted to lower limits by the bank.
- **Variability and Outliers:** Both groups show a large number of outliers, but their presence in the default group is particularly interesting. It indicates a higher variability in financial behavior among those who default—meaning some high-limit customers still fail to pay, though they are fewer in number



Payment Behavior Analysis

To understand what drives the model's predictions, we analyzed how repayment history and payment amounts relate to defaulting.

1. Total Delay vs. Default

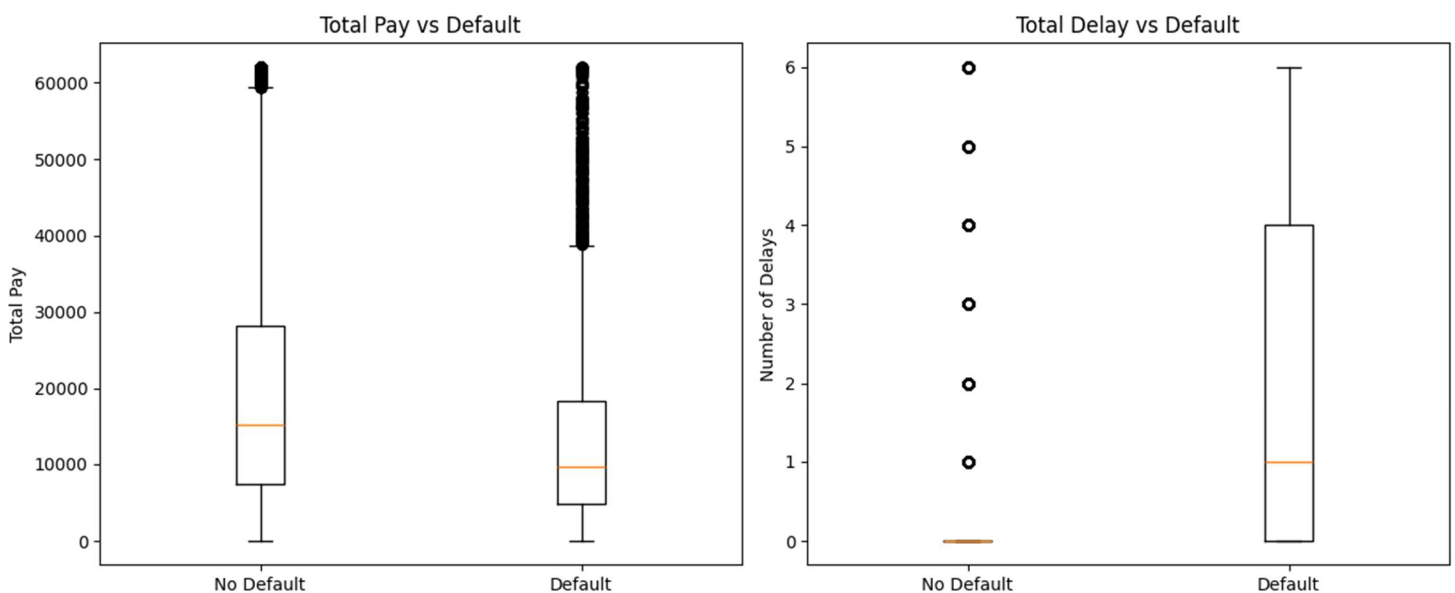
The analysis of repayment delays reveals a clear pattern:

- **Reliability:** For the "No Default" group, the median delay is at zero. This shows that most reliable customers never miss a payment.
- **Warning Signs:** The "Default" group shows much higher boxes, representing frequent delays of **1 to 4 months**. This is the clearest indicator of risk; even a single late payment significantly increases the likelihood of falling into the default category.

2. Total Payment vs. Default

The amount paid monthly also serves as a strong predictor:

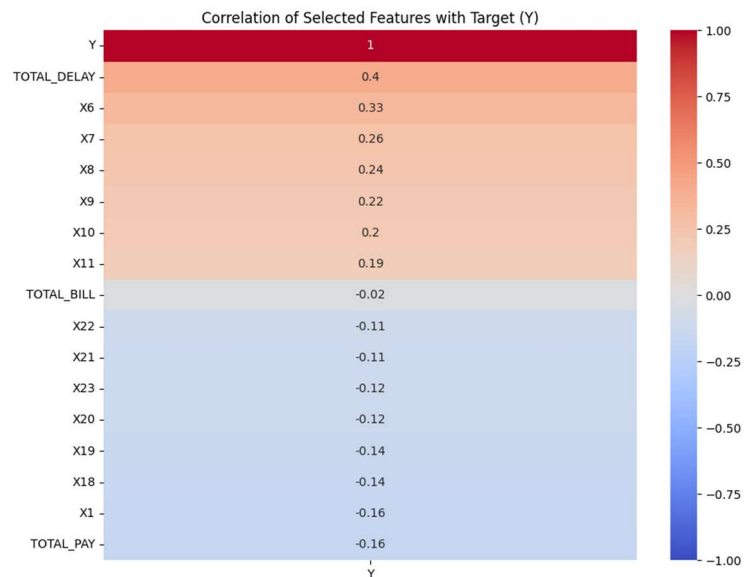
- **Payment Magnitude:** Non-defaulters generally make larger monthly payments. In contrast, the "Default" group typically pays smaller amounts, which acts as an early warning sign for banks.
- **Complexity of Debt:** Interestingly, some high-paying customers still default (shown as outliers). This suggests that even large payments might not be enough if the customer's total debt is excessively high.



Some features might be redundant or have a very weak relationship with the target variable. To improve the model's accuracy and reduce complexity, we performed a **Correlation Analysis**.

Based on our findings, we selected the features with the highest correlation to the target variable (Y). The selected features are:

- **Credit Limit & Historical Payments:** ['X1', 'X18', 'X19', 'X20', 'X21', 'X22', 'X23']
- **Repayment Status:** ['X6', 'X7', 'X8', 'X9', 'X10', 'X11']
- **Engineered Features:** ['TOTAL_BILL', 'TOTAL_PAY', 'TOTAL_DELAY']



By focusing on these specific variables, we ensured that the model learns from the most impactful data, leading to a more reliable prediction of whether a customer will default or not.

Implementing First classifier:

For our first choice of classifiers we chose Logistic Regression which outputs probabilities for both classes using sigmoid function

```
#separating the target variable (Y) from the features (X)
y = df["Y"]
X = df[['X1', 'X6', 'X7', 'X8', 'X9', 'X10', 'X11',
        "TOTAL_BILL", "TOTAL_PAY", "TOTAL_DELAY"]]

from sklearn.model_selection import StratifiedKFold, train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import KFold, StratifiedKFold

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

We used **train_test_split()** to split data for train and test , we used 80% for training and 20 for test% , **stratify = y** was used to preserve class distribution in the train and test dataset.

```

pipeline = Pipeline([
    ('model', LogisticRegression(random_state=42, class_weight='balanced'))
])
param_grid = {
    'model__max_iter': [1000, 10000, 100000],
    'model__C': [0.01, 0.1, 1, 10, 100],
    'model__penalty': ['l1', 'l2'],
    'model__solver': ['saga']
}

cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)
grid_search = GridSearchCV(estimator=pipeline, param_grid=param_grid, cv=cv, scoring='f1')
grid_search.fit(X_train, y_train)

```

Used LogisticRegression Class from sklearn to initialize the model with hyperparameter **class_weight = 'balanced'** to give different weights based on class distribution so this will help with the class imbalance.

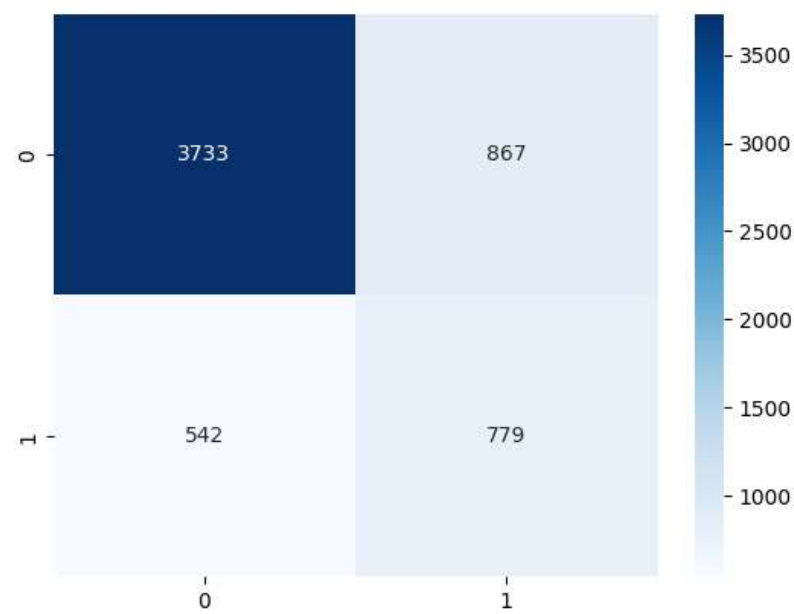
used GridSearch to find the most optimal parameters for this model and dataset, and used cross validation with 10 splits. This will divide data into 10 splits train on 9 and validate on the one left. This was the optimal parameters we obtained:

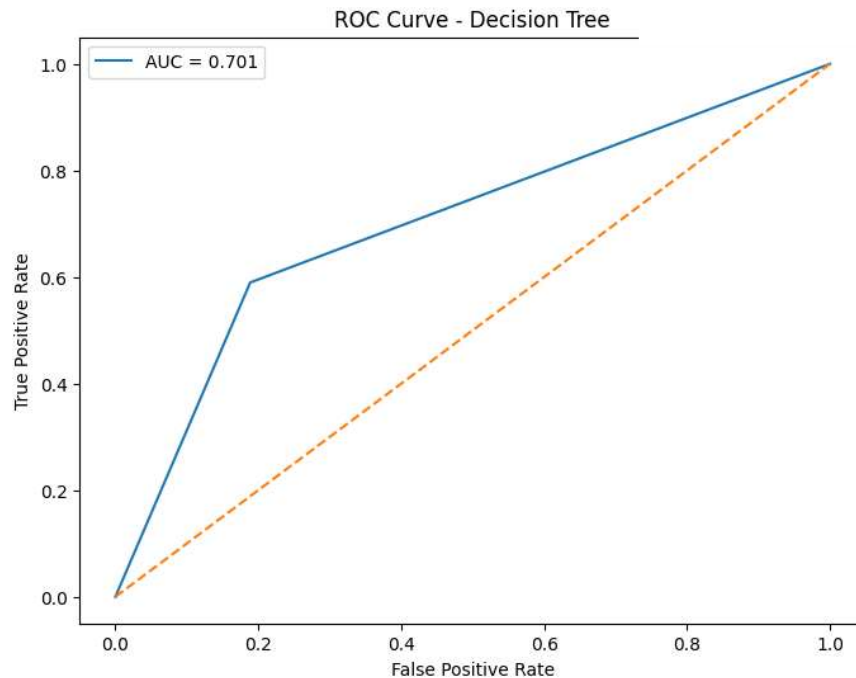
LogisticRegression
LogisticRegression(C=0.01, class_weight='balanced', max_iter=1000, penalty='l1', random_state=42, solver='saga')

L1 penalty is a penalty on the absolute value of the coefficients, so weak features may have near zero or zero coefficients. C parameter is used to control regularization.

Here are the results of the model: used threshold 0.7.

Classification Report:					
	precision	recall	f1-score	support	
0.0	0.87	0.81	0.84	4600	
1.0	0.47	0.59	0.53	1321	
accuracy			0.76	5921	
macro avg	0.67	0.70	0.68	5921	
weighted avg	0.78	0.76	0.77	5921	
ROC-AUC Score: 0.7530					
F1-Score: 0.5251					
Accuracy: 0.7620					
Precision: 0.4733					
Recall: 0.5897					





As we can see the Logistic Regression model performs very badly on the minor class, and gives overall f1 score of 0.525 which is low. So, we conclude that logistic regression is not suitable for this dataset at all due to its low performance on the minority class.

2nd Model Shallow Neural Network

Used 2 hidden layers, first one is 128 neuron and second one is 64 neurons both have activation function Relu. Input layer is 9, I removed one feature due to low correlation with the output, used **Batch Normalization** and **dropout** layers to avoid overfitting and make model converge faster, output is one neuron with a sigmoid activation function.

Batch Normalization is a technique used to stabilize and accelerate the training of deep neural networks by re-centering and re-scaling the inputs to each layer.

During training, a Dropout layer randomly "turns off" a percentage of neurons in every pass.

- **Randomness:** Because a different set of neurons is disabled each time, the network cannot rely on any single neuron to pass information.
- **Redundancy:** Every neuron is forced to learn useful features independently.
- **The Result:** At the end of training, you have a more robust, "well-rounded" model that generalizes much better to new data.

```

model_nn = Sequential([
    layers.Dense(128, activation='relu', input_shape=(9,)),
    layers.BatchNormalization(),
    layers.Dropout(0.3),
    layers.Dense(64, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.3),
    layers.Dense(1, activation='sigmoid')
])

```

```

model_nn.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss='binary_crossentropy',
    metrics=[
        metrics.BinaryAccuracy(name='accuracy'),
        metrics.AUC(name='auc'),
        metrics.Precision(name='precision'),
        metrics.Recall(name='recall'),
        metrics.AUC(name='prc', curve='PR')
    ]
)

```

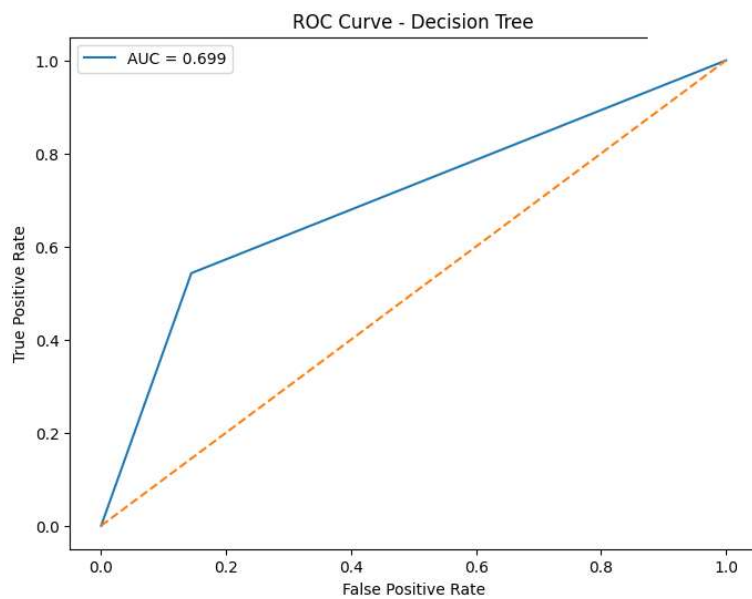
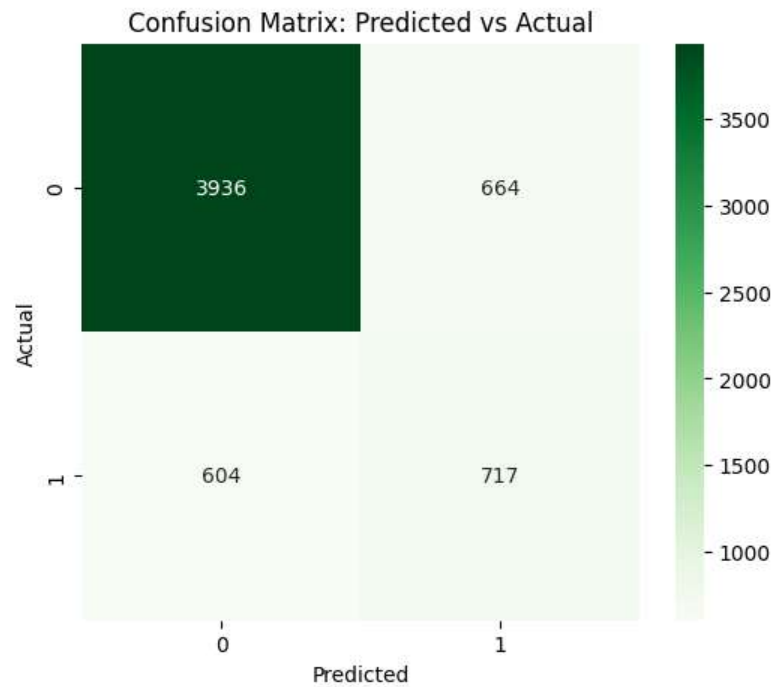
Used Adam optimizer and loss criteria will be binary cross entropy as our data is binary classification. We used accuracy, auc, precision and recall as a judge metrics.

```

# 3. adjusting weights to count for class imbalance
strong_class_weight = {0: 0.8, 1: 4}
history = model_nn.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=150,
    class_weight=strong_class_weight
)

```

Trained model on a custom weight due to huge class imbalance we have in our data, trained model on 150 epochs.



--- Credit Default Model Metrics ---				
	precision	recall	f1-score	support
No Default	0.87	0.86	0.86	4600
Default	0.52	0.54	0.53	1321
accuracy			0.79	5921
macro avg	0.69	0.70	0.70	5921
weighted avg	0.79	0.79	0.79	5921

As we can see we have a slightly better model but still not good. So still our model can't learn the features of the weak class due to lack of data

3rd Model Decision Trees

A Decision Tree is a supervised learning algorithm that makes predictions by breaking down a dataset into smaller and smaller subsets based on specific features. It works through a flowchart-like structure and at the end it tells us which class does this data belongs to

Parameter	Function	Why it matters
criterion	The mathematical formula used to measure the quality of a split.	Gini is faster to compute; Entropy is slightly more intensive but can sometimes produce more balanced trees.
max_depth	Limits how many levels deep the tree can grow.	Prevents overfitting . A tree that is too deep memorizes the data; a tree too shallow fails to learn patterns.
min_samples_split	The minimum number of data points a node must have before it is allowed to split.	Higher values prevent the tree from creating branches based on very small, noisy groups of data.
min_samples_leaf	The minimum number of data points required to be at a "leaf" (end) node.	It smoothes the model. It ensures that no prediction is based on just one or two isolated samples.
max_features	The number of features to consider when looking for the best split.	None uses all features; sqrt uses a random subset. Using fewer features helps prevent overfitting and speeds up training.
ccp_alpha	A complexity parameter used for Cost Complexity Pruning .	A value greater than 0 "clips" off weak branches after the tree is built to make it simpler and more robust.

Parameter	Function	Why it matters
class_weight	Adjusts the importance of each class (0 and 1).	Vital for imbalanced data . Setting it to 'balanced' or {0:1, 1:3} tells the model to pay 3x more attention to the minority class.

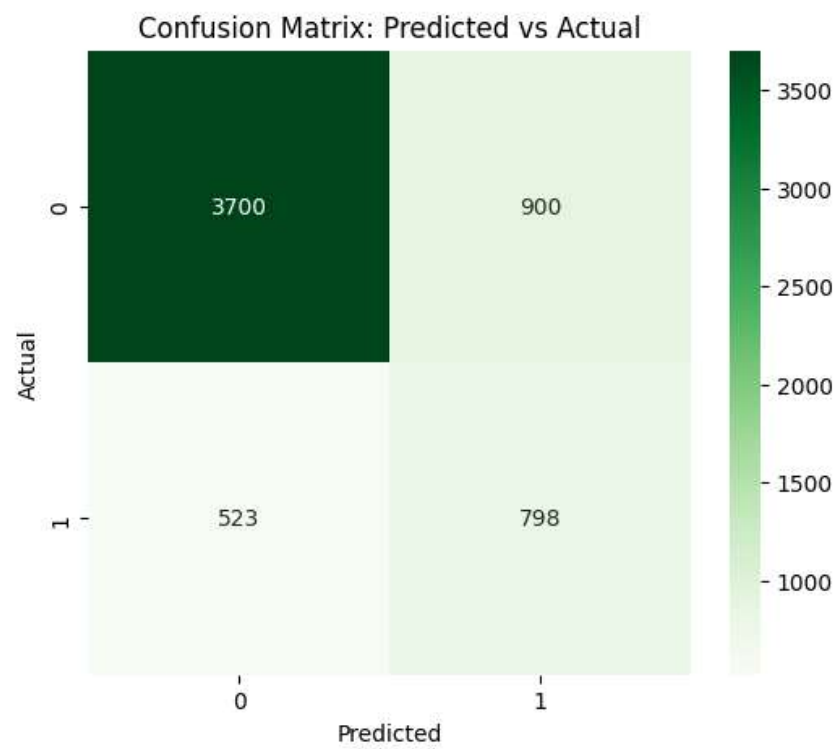
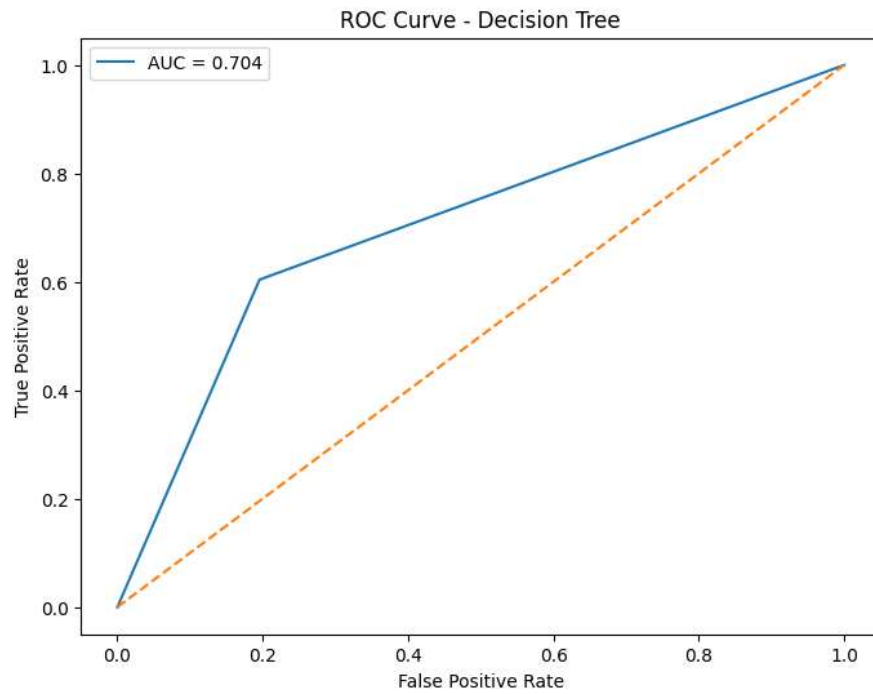
```

params = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [4, 6, 8, 10, 12],
    'min_samples_split': [10, 20, 50, 100],
    'min_samples_leaf': [2, 5, 10, 20],
    'max_features': [None, 'sqrt'],
    'ccp_alpha': [0.0, 0.0001, 0.001],
    'class_weight': [
        None,
        'balanced',
        {0:1, 1:2},
        {0:1, 1:3},
    ]
}

grid = GridSearchCV(
    DecisionTreeClassifier(
        class_weight='balanced',
        random_state=42
    ),
    params,
    scoring='f1',
    cv=5,
)

```

I used GridSearch to find the most optimal parameters and used 5 splits for the Cross validation



```
... --- Credit Default Model Metrics ---
```

	precision	recall	f1-score	support
No Default	0.88	0.80	0.84	4600
Default	0.47	0.60	0.53	1321
accuracy			0.76	5921
macro avg	0.67	0.70	0.68	5921
weighted avg	0.79	0.76	0.77	5921

Results are still in the same range as the other models, just slightly better recall for class 1.

My conclusion is that the problem is from the dataset itself not the architecture of our models.